

Student 29 **NITHISH RAAJU V** 192411016RD

5 / 9 submitted

Q1. Banking Transaction System – Exception Handling

CODE

```
import java.util.*;
public class main{
    public static void main(String[] args){
        Scanner sc=new Scanner(System.in);
        try{
            double balance=sc.nextDouble();
            double amount=sc.nextDouble();
            if (amount<=0)
                throw new IllegalArgumentException("Invalid withdrawal amount");
            if (amount>balance)
                throw new InsufficientBalanceException("Insufficient balance");
            System.out.println("Withdrawal Successful");
            System.out.println("Remaining balance:"+ (balance-amount));

        }catch(InputMismatchException e){
            System.out.println("Invalid numeric input");
        }catch(InsufficientBalanceException | IllegalArgumentException e){
            System.out.println(e.getMessage());
        }finally{
            System.out.println("Transaction completed");
        }
    }
}

class InsufficientBalanceException extends Exception{
    InsufficientBalanceException(String msg){
        super(msg);
    }
}
```

INPUT

10000 3000

OUTPUT

Withdrawal Successful
Remaining balance:7000.0
Transaction completed

Q2. Hospital Patient Registration – User-Defined Exception

CODE

```
import java.util.*;
public class Main{
    public static void main(String[] args){
        Scanner s=new Scanner(System.in);
        try{
            String name=s.next();
            int age=s.nextInt();
            double temp =s.nextDouble();
            if (age<18)
                throw new InvalidAgeException("InvalidAgeException");
            if (temp<95 || temp>100)
                throw new InvalidTemperatureException("InvalidTemperatureException");
            System.out.println("Patient registered successfully");

        }catch(InvalidAgeException e){
            System.out.println(e.getMessage());
        }catch(InvalidTemperatureException e){
            System.out.println(e.getMessage());
        }finally{
            System.out.println("Registration completed");
        }
    }
}

class InvalidAgeException extends Exception{
    InvalidAgeException(String m){
        super(m);
    }
}

class InvalidTemperatureException extends Exception{
    InvalidTemperatureException(String m){
        super(m);
    }
}
```

INPUT

Arun 35 98.6

OUTPUT

Patient registered successfully
Registration completed

Q3. E-Commerce Order Processing – Built-in Exceptions

CODE

```
import java.util.Scanner;
import java.util.InputMismatchException;

class Main
{
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);

        double []price={100,200,300};
        try{
            int index=sc.nextInt();
            int qty=sc.nextInt();
            if(qty<=0){
                throw new IllegalArgumentException("Invalid quantity");
            }
            System.out.println("Total : "+price[index]*qty);
        }
        catch(ArrayIndexOutOfBoundsException e){
            System.out.println("Invalid product index");
        }
        catch(IllegalArgumentException e){
            System.out.println(e.getMessage());
        }
    }
}
```

INPUT

1 2

OUTPUT

Total : 400.0

Q4. Railway Ticket Booking – Multithreading and Synchronization

— Not Submitted —

Q5. Bank Account – Concurrent Withdrawals

— Not Submitted —

Q6. Online Food Delivery – Multiple Threads and Execution Order

CODE

```
class Main{
    public static void main(String[] args) throws Exception{
        Thread order=new Thread()->System.out.println("Order Processing");
        Thread payment=new Thread()->System.out.println("Payment Processing");
        Thread delivery=new Thread()->System.out.println("Delivery Assignment");
        order.start();
        order.join();
        payment.start();
        payment.join();
        delivery.start();
        delivery.join();
        System.out.println("Order completed");
    }
}
```

OUTPUT

```
Order Processing
Payment Processing
Delivery Assignment
Order completed
```

Q7. Hospital Emergency System – Thread Priority

CODE

```
class Main{
    public static void main(String[] args) throws Exception{
        Thread emergency=new Thread() -> System.out.println("Emergency");
        Thread general=new Thread() -> System.out.println("General");
        Thread billing=new Thread() -> System.out.println("Billing");
        emergency.setPriority(10);
        general.setPriority(5);
        billing.setPriority(1);
        emergency.start();
        general.start();
        billing.start();
        emergency.join();
        general.join();
        billing.join();
        System.out.println("All processing completed");
    }
}
```

OUTPUT

```
General
Billing
Emergency
All processing completed
```

Q8. Manufacturing Plant – Producer-Consumer Problem

— Not Submitted —

Q9. Online Banking – Exception Handling with Multiple Threads

— Not Submitted —